E-COMMERCE BACKEND SYSTEM USING PYTHON AND DJANGO FRAMEWORK

By
YEUKULA PADMAVATHI
KATIKE VINOD KUMAR

PALLE TECHNOLOGIES
Software Training Institute
HSR Layout, Bengaluru
560068

CERTIFICATE

This is to certify that this Project Report is the work of Yerukula Padmavathi and Katike Vinod Kumar who carried out the project entitled “E-COMMERCE BACKEND SYSTEM USING PYTHON AND DJANGO FRAMEWORK”.

Internal Guide
S.Ragunath
Technical Trainer
Palle Technologies

DECLARATION

I, Yerukula Padmavathi and Katike Vinod Kumar hereby declare that the project report entitled **"E-COMMERCE BACKEND SYSTEM USING DJANGO FRAMEWORK"** done by me under the guidance of **S.Ragunath sir** .

Date:20-Dec-2024
Place: Bangalore

SIGNATURE OF THE CANDIDATE
Yerukula Padmavathi

ACKNOWLEDGEMENT

I am pleased to acknowledge my sincere thanks to **Palle Technologies** for their kind encouragement in doing this project and for completing it successfully. I am grateful to them.

I convey my thanks to **S.Ragunath** sir for providing me necessary support and and details at the right time during the progressive reviews.

I would like to express my sincere and deep sense of gratitude to my Project Guide Ragunath sir for his valuable guidance, suggestions and constant encouragement paved way for the successful completion of my project work.

ABSTRACT

In today's fast-changing business environment, it's extremely important to be able to respond to client needs in the most effective and timely manner. If your customers wish to see your business online and have instant access to your products or services. E-commerce is fast gaining ground as an accepted and used business paradigm. More and more business houses are implementing web sites providing functionality for performing commercial transactions over the web. It is reasonable to say that the process of shopping on the web is becoming commonplace. These types of online shops have become part of our daily lives. It helps organizations to reduce the cost to create process, distribute, retrieve and manage the paper-based information by digitalizing the information. The general purpose of E-Commerce store where products can be bought from the comfort of home through the Internet.

TABLEOF CONTENTS

Chapter No.	TITLE	Page No.
	ABSTRACT	v
	LISTOFFIGURES	ix
1	INTRODUCTION	1
	1.1 OVERVIEW	1
2	LITERATURESURVEY	2
3	AIMAND SCOPEOFTHEPROJECT	5
	3.1.1 AIMOFTHEPROJECT	5
	3.1.2 SCOPEANDOBJECTIVE	5
	3.1.3 SYSTEMREQUIREMENTS	6
	3.1.4 HARDWAREREQUIREMENTS	6
	3.1.5 SOFTWAREREQUIREMENTS	6
	3.1.6 SOFTWAREUSED	7
	3.1.7 PYTHONLANGUAGE	7
	3.1.8 FEATURESPYTHON	7
	3.1.9 DJANGOFRAMEWORK	8
	3.1.10 CHARACTERISTICSOFDJANGO FRAMWORK	8
	3.1.11 HTML	8
	3.1.12 CSS	9
	3.1.13 JAVASCRIPT	9
	3.1.14 BOOTSTRAP	9
	3.1.15 APPLICATIONDEVELOPMENTPLATFORM	11
	3.1.16 VS CODE	10
4	EXPERIMENTALORMATERIALMETHODS	10
	4.1 DESIGNMETHODOLOGY	11
	4.1.1 EXISTINGSYSTEM	11
	4.1.2 PROPOSEDSYSTEM	11
	4.2 APPLICATIONDESCRIPTION	12

4.2.1 HOMEPAGE 12

4.2.2 REGISTERPAGE 12
4.2.3 LOGINPAGE 12
4.2.4 PRODUCTVIEWPAGE 12
4.2.5 CARTPAGE 12
4.2.6 CHANGEPASSWORDPAGE 12
4.2.7 CONTACTUSPAGE 13
4.2.8 TRACKORDERPAGE 13
4.3 SYSTEMARCHITECTURE 13
4.4 USECASEDIAGRAM 14
4.4.1 CUSTOMERSIDE 14
4.4.2 ADMINSIDE 14
4.5 IMPLEMENTATION 15
4.5.1 CREATINGTHEVIRTUALENVIRONMENT 15
4.5.2 CREATINGVARIOUSAPPS 16
4.5.3 CREATINGMODELS 16
4.5.4 CREATINGASUPERUSER 17
4.5.5 CREATINGVIEWSFORTHEMODELS 17
4.5.6 HOWDOESDJANGOWORK? 18
4.6 APPLICATIONFUNCTIONALITIES 19
4.6.1 USERCREATION 19
4.6.2 COOKIES 19
4.6.3 SEARCHFUNCTIONALITY 19
4.6.4 CARTFUNCTIONALITY 20
4.6.5 ORDERFUNCTIONALITY 20
4.6.6 CHANGEPASSWORDFUNCTIONALITY 20
4.6.7 CONTACTUSFUNCTIONALITY 21
4.7 TESTING 21
5 RESULTSANDPERFORMANCE ANALYSIS 22
5.1 ADMINISTRATIONPANEL 22
5.2 HOMEPAGEBEFORELOGIN 22
5.3 HOMEPAGEAFTERLOGIN 23
6 CONCLUSIONANDFUTUREWORK 25

6.1 CONCLUSION 25
6.2 FUTUREWORKS 25

APPENDIX 28
A.SOURCECODE 28

LISTOFFIGURES

FIGURE NO.	FIGURENAMES	PAGE NO.
4.3	WebApplicationArchitecture	13
4.4.1	Customer Side	14
4.4.2	AdminSide	14
4.5.1	DjangoWebserver	16
5.1	AddproductfeatureinDjangoAdministrationPanel	22
5.2	Homepagewithaddedproducts	23
5.3	Home pageafterlogin	24

CHAPTER–1

1. INTRODUCTION

1.1 OVERVIEWOFTHEPROJECT

Today modern Web applications have grown into complex distributed applications.Theworldwideexpansionofthe internethasconsiderablycontributed to the change of trade and store exchanges. Thus, today modern web applications have grown into complex distributed applications.

E-commerce(electroniccommerceorEC)isthebuyingandsellingofgoods and services, or the transmitting of funds or data, over an electronic network, primarily the internet. These business transactions occur either as business-to-business,business-to-consumer,consumer-to-consumerorconsumer-to-business.

Electronic commerce (e-commerce) is a fairly new idea, and it is very common practice nowadays for businesses to conduct trade over the Internet. It additionally makes the use of regular technological maintenance to ensure the smooth workingof online store sites, monetary exchanges, just as everything to do with giving and delivering items.

Itcansimplybedefinedaspurchasingandadditionallysellingitemsthrough the internet and is ordinarily associated with online shopping

Here, we are going to develop an e-commerce application, which has penetrated into people's daily life, and the e-commerce market is becoming more and more competitive.

CHAPTER–2

2. LITERATUREREVIEW

1. Django:WebDevelopmentSimple&Fast

Django is a web application framework which is open source and written in the Python language. It uses MVT design structure (MVT stands for Model View Template). Due to its rapid development feature. Django is very demanding in the current Market. It takes less time to build any kind of application. Why we say this Model ViewTemplatebecausethisframeworkwill workbaseduponthemodelasa database and view as a controlling functionality and template will work as a user side for communication interaction. The Django model will work as database management, we use two main commands like: - python manage.py make migrationsDjangowill deductthechangesin models.pyfileandreadytosenddata into the sqlite3 (choose any database). Then we make python manage.py migrate. thentheDjangosystemwillsaveallchangesinhisdatabasesystem.Thenwemake one more command Python manage.py run server at the end this will start our project and gives us the localhost address for the project running locally. And views.py file will handle the request for the project to the API's call to template management in requests. we can write the views in the form of python functions.

2. ElectronicCommerce:AStudyonBenefitsandChallengesinan Emerging Economy

Information Technology has been playing a vital role in the future development of financial sectors and the way of doing business in an emerging economy like Bangladesh. Increased use of smart mobile services and internet as a new distribution channel for business transactions and international trading

requires more attention towards e-commerce security for reducing the fraudulent activities. The advancement of Information and Communication technology has broughtalotofchangesinallspheresofdailylifeofhumanbeing.Ecommercehas a lot of benefits which add value to customer's satisfaction in terms of customer convenience in any place and enables the company to gain more competitive advantage over the other competitors. This study predicts some challenges in an emerging economy.

3. AStudyonimpactofE-Commerce onIndia'scommerce

E-commerce involves an online transaction. E-commerce provides multiple benefitstotheconsumersinformofavailabilityofgoodsatlowercost,widerchoice and saves time. The general category of ecommerce can be broken down into two parts: E-Merchandise & E-finance. Many companies, organizations, and communities in India are doing business using E-commerce and also are adopting M-commerce for doing business. Ecommerce is showing tremendous business growth in India. Increasing internet users have added to its growth. Despite being the second largest user base in world, only behind China (650 million, 48% of population), the penetration of e-commerce is low compared to markets like the United States (266 M, 84%), or France (54 M, 81%), but is growing at an unprecedented rate, adding around 6 million new entrants every month. The industryconsensusisthatgrowthisataninflectionpoint.India'secommercemarket was worthabout$3.9billionin2009,itwentupto$12.6billionin2013.In2013,the e-retailsegmentwasworthUS$2.3billion.About70%ofIndia'se-commercemarket is travel related. According to Google India, there were 35 million online shoppers in India in 2014 By 2020, India is expected to generate $100 billion online retail revenue out of which $35 billion will apparel sales are set to grow four times in comingyears.Thispaperisoutcomeofareviewofvariousresearchstudiescarried out on Impact of E-commerce on Indian.

4. DjangoBasedWebApplicationtoEmpowerSkilledPeople

ThispaperfocusesondevelopmentofwebapplicationusingDjango.Django is a modern Python web framework that redefined web development in the Python world.Afullstackapproach,pragmaticdesign,andsuperbdocumentationaresome of the reasons for its success. Django, an open-source Python web framework that saves time and makes web development fun. Django follows the Model-View-Controller (MVC) architectural pattern. Its goal is to ease the creation of complex, database-driven websites. Django emphasizes reusability and “pluggability” of components, rapid development, andthe principle of DRY(Don't Repeat Yourself). Python is used throughout, even for settings, files, and data models. Technological Implementations in the field of job search has helped skilled people as well as people who wants skilled workers in very important ways. The availability of all job searching sites helps the skilled people a lot in their day-to-day life. The paper illustrates a website model with the help of which skilled people can be able to update their skills and people who wants skilled workers can find them from the same digital platform as well. The project is developed on Django Framework; the backend development is in Python, Jinja2 and SQLite. The frontend consists of HTML, CSS and JavaScript. The project developed is highly efficient, user friendly and simple.

CHAPTER–3

3. AIMANDSCOPEOFTHEPRESENTATION

3.1 AIMOFTHEPROJECT

The reason behind our motivation is the current trend of web application integration and interactive features. The trends of online shopping came into existence in the early 90's. Today, therearemany online shopping system in place but there are many problems such as hands on experience, fraud and security concern,privacy,lackoffullcostdisclosure,productreviewsand soon.Ourproject istolooktothecurrentproblemsandtopresentonone-stop-shoppingplatformthat is beneficial for both customers and sellers.

3.2 SCOPEANDOBJECTIVE

In our platform, the users can view various products, if they wish to buy any then they have to register and then login to add that product inside their cart. The usersorcustomerscanalsoreadthereviewsaboutaproductpostedbyotherusers before buying it.

The users can add how many products he/she wants to their shopping cart. Then the users are able to set the quantity of each added product inside the cart. Finally, while checkout the users can give their address and the mode of payment and place the respective order. Then the admin can see the customer details with his/her order details and the address where the order should be delivered.

Online Shopping is a very popular in web development. It is mostly used as a business modelto earn capital The primaryobjective of this project is to develop an E-commerce web application to reach maximum customers at the right time to increase sales and profitability of the business including buying and selling goods, transmitting funds or data over the internet.

HeretheAdmincanuploadtheirproductdetailsinwebsiteandthecustomers canvisittheHomePageandcheckwhethertheproductisavailableornot.Any

member can register and view available products. Only registered member can purchase multiple products regardless of quantity.

The customers can add or view reviews about the products. The customers can add different products to cart and can change the quantity. To place the order, the customer can proceed checkout where they enter the details like address, contact number and selecting the mode of payment. After placing the order, the customer will get an Order ID. With the Order ID, they can track the details of the order intheTrackOrder Page.Ifthecustomerfacesanydifficulties or issues inthe website,aContactUs pageis availableto contactAdminforqueriesandreportthe problem

3.3 SYSTEMREQUIREMENTS

3.3.1 HardwareRequirements

The selection of hardware is very important in the existence and proper working of any software. When selecting the hardware, the size and requirements arealsoimportanttorunthesoftware.Theminimumhardwarerequirementsareas follows:

- Processor:IntelCOREi3
- RAM:4GB
- Diskspace:minimum256GB

3.3.2 SoftwareRequirements

- OperatingSystem(Windows,MacOS).
- Python,HTML,CSS,JavaScript,Bootstrap.
- DjangoFrameworkandSQLitedatabase(whichcomesbydefaultwith Django).
- AnInternetBrowser(GoogleChrome,MicrosoftEdgeetc).
- CodeEditor(VisualStudiocode,PyCharm).
- The package manager PIP (pip is a python package-management system written in Python used to install and manage software packages).

3.4 SOFTWAREUSED

3.4.1 PythonLanguage

Python language is a high-level, dynamically typed one that is among the most popular general-purpose programming languages. Python is an Interpreted, object-oriented, and high-level programming language. It is called an interpreted language as its source code is compiled to bytecode which is then interpreted. Python's features, among other things, are what make it popular. For instance, it supports dynamic typing and dynamic binding.

3.4.2 FeaturesofPython

- Pythonisopensource.Youcandownloaditforfreeanduseitinyour application. You can also read and modify the source code.
- ThePython frameworkalsohasmodules andpackages, whichfacilitates code reusability.
- It provides rich data types and easier to read syntax than any other programming
- languages
- It is a platform independent scripted language with full access to operating system API's
- Comparedtootherprogramming languages,itallowsmorerun-timeflexibility
- ItincludesthebasictextmanipulationfacilitiesofPerl andAwk
- AmoduleinPythonmayhaveone ormoreclassesandfree functions
- LibrariesinPythonsarecross-platformcompatiblewithLinux,Macintosh, and Windows
- Forbuildinglargeapplications,Pythoncanbecompiledtobyte-code
- PythonsupportsfunctionalandstructuredprogrammingaswellasOOP
- ItsupportsinteractivemodethatallowsinteractingTestinganddebuggingof snippets of code
- In Python,sincethereis no compilationstep,editing, debugging and testing is fast.

3.4.3 DjangoFramework

Django is a high-level Python web framework that encourages rapid developmentandclean,pragmaticdesign.Builtbyexperienceddevelopers,ittakes care of much of the hassle of web development, so you can focus on writing your app without needing to reinvent the wheel. It'sfree and open source.

3.4.4 CharacteristicsofDjangoFramework

- Django was designed to help developers take applications from concept to completion as quickly as possible.
- Itincludesdozensofextrasyoucanusetohandlecommonwebdevelopment tasks. Django takes care of user authentication, content administration, site maps,RSSfeeds,andmanymoretasks—rightoutofthebox.
- Ittakessecurityseriouslyandhelpsdevelopersavoidmanycommonsecurity mistakes, such as SQL injection, cross-site scripting, cross-site request forgery and clickjacking. Its user authentication system provides a secure way to manage user accounts and passwords.
- Some of the busiest sites on the planet use Django's ability to quickly and flexibly scale to meet the heaviest traffic demands.
- Companies,organizationsandgovernmentshaveusedDjangotobuildall sortsofthings—fromcontentmanagementsystemstosocialnetworksto scientificcomputing platforms.

3.4.5 HTML

Hypertext Markup Language (HTML) is the standard markup language for creatingwebpagesandwebapplications.WithCascadingStyle Sheets(CSS) and JavaScriptitformsatriadofcornerstonetechnologiesfortheWorldWideWeb.Web browsers receive HTML documents from a webserver or from local storage and render them into multimedia web pages. HTML describes the structure of a web pagesemanticallyandoriginallyincludedcuesfortheappearanceofthedocument. HTML can embed programs written in a scripting language such as JavaScript which affect the behavior and content of web pages.

3.4.6 CSS

Cascading Style Sheets (CSS) is a style sheet language used for describing the presentation of a document written in a markup language. Although most often used to set the Visual style of web pages and user interfaces written in HTML and XHTML, the language can be applied to any XML document, including plain XML, SVG and XUL, and is applicable to rendering in speech, or on other media. Along withHTMLandJavaScript,CSSisacornerstonetechnologyusedbymostwebsites tocreatevisuallyengagingwebpages,userinterfacesforwebapplications,anduser interfaces for many mobile applications. CSS is designed primarily to enable the separationofpresentationandcontent,includingaspectssuchasthelayout,colors, andfonts.Thisseparationcanimprovecontentaccessibility,providemoreflexibility andcontrolinthespecificationofpresentationcharacteristics,enablemultipleHTML pagestoshareformattingbyspecifyingtherelevantCSSin aseparate.cssfile,and reduce complexity and repetition in the structural content.

3.4.7 JavaScript

JavaScript is a high level, dynamic, untyped, and interpreted programming language. It has been standardized in the ECMAScript language specification. Alongside HTML and CSS, it is one of the three essential technologies of World WideWeb contentproduction;the majorityofwebsitesemployitanditissupported by all modern web browsers without plug-ins. JavaScript is prototype-based with first-class functions, making it a multi-paradigm language, supporting object- oriented, imperative, and functional programming styles. It has an API for working with text, arrays, dates andregular expressions, but does not includeany I/O, such as networking, storage or graphics facilities, relying for these upon the host environment in which it is embedded.

3.4.8 Bootstrap

Bootstrap is a free and opensource CSS framework directed at responsive, mobilefirstfront-endwebdevelopment.ItcontainsCSSand(optionally) JavaScript-based design templates for typography, forms, buttons, navigation, and other interface components.

3.5 APPLICATIONDEVELOPMENTPLAFTORM

3.5.1 VSCode

Visual Studio Code is a source-code editor made by Microsoft for Windows, Linux and macOS. Features include support for debugging, syntax highlighting, intelligent code completion, snippets, code refactoring, and embedded Git. Users canchangethetheme,keyboardshortcuts,preferences,andinstallextensionsthat add additional functionality.

It can be used with a variety of programming languages, including Java, JavaScript, Go, Node.js, Python and C++.

CHAPTER–4

4. EXPERIMENTALORMATERIALMETHODS

4.1 DESIGNMETHODOLOGY

4.1.1 ExistingSystem

E-commerce is growing pretty fast, however, there is no standardization for payment system, resulting unreliability of online payment. There is still another problem following setting up an online store with the web site, the problem of marketing may arise. Another issue, to control the trustworthy of the web site, all shop owners have to register their profiles to the web site, thus it will assure that there is no fraud in online trading on the web site. As the result, the web site will retain its reputation in the market. With this strict rule, the web site can control the quality and trust of its customers. This is very important issue for such a large web site.

4.1.2 ProposedSystem

The web site, designed as an online shopping center, is separated into two parts: back end and front-end parts. This part provides facility for each store owner to edit and modify information in his own store. Providing validation check for member and store identification, the back-end system can securely protect users' proprietaryinformation.Inaddition,allpageviewsemploysessionvariablestodeter manuallydefinedvariablesbyusers.Applying userfriendlyapproach,andfocusing on web programming inexperience, the user can effortlessly manage his back-end information. Inside the back end, users can control and view all store information. Besides that, using content management design, the back-end part encompasses with these modules: admin panel, home, register, login, product view, cart, checkout, change password, contact us and track order

4.2 APPLICATIONDESCRIPTION

4.2.1 HomePage/Landingpage:

In this page, all the products are displayed on the home screen with the image, name and price of the products. Two buttons are also created, one is for adding theiteminsidethecartandtheotheristoviewtheproduct.Ifthecustomers are not logged in then they are not able to add the item inside their cart. They can just see all the products. On the add to cart button, login to add the item will be written. The customers can directly search for the product that they want on the search option given on the navigation bar.

4.2.2 RegisterPage:

In this page, users can register themselves, in order to view the product details and place an order.

4.2.3 Loginpage:

Here,userscantypetheirusernameand passwordtologin.

4.2.4 ProductViewpage:

After clickingontheviewbutton,thecustomerscanviewthe specificproduct with their key features and reviews. After reading the key features and the reviews about the product, the customer can buy the product by clicking on the add to cart button. The customer who has brought that specific product can write their review about the product which the other customer is able to read.

4.2.5 Cartpage:

By clicking on the shopping cart icon on the navigation bar, customers can see all the added items in the cart. The users can then increase or decrease the quantity of the products according to their requirements.

4.2.6 ChangePasswordpage:

All the customers can change their password by going to the change password option.

4.2.7 ContactUspage:

Thecustomerscanasktheirqueriesorcancontactusbyfillingasmallform. There are two different forms one for the logged in users and others who haven't registered themselves but want to contact us. If the user is a logged in user, then theuserjust,havetowritethemessagedirectlyelsetheuserneedstofirstgivehis name, email and phone before contacting.

4.2.8 TrackOrderpage:

Afterplacingtherequiredorder,youwillgetanorderid.Thatidcanbefurther used for tracking the order. In the track order menu, you have to give your order id for viewing the status of the order.

4.3 SYSTEMARCHITECTURE

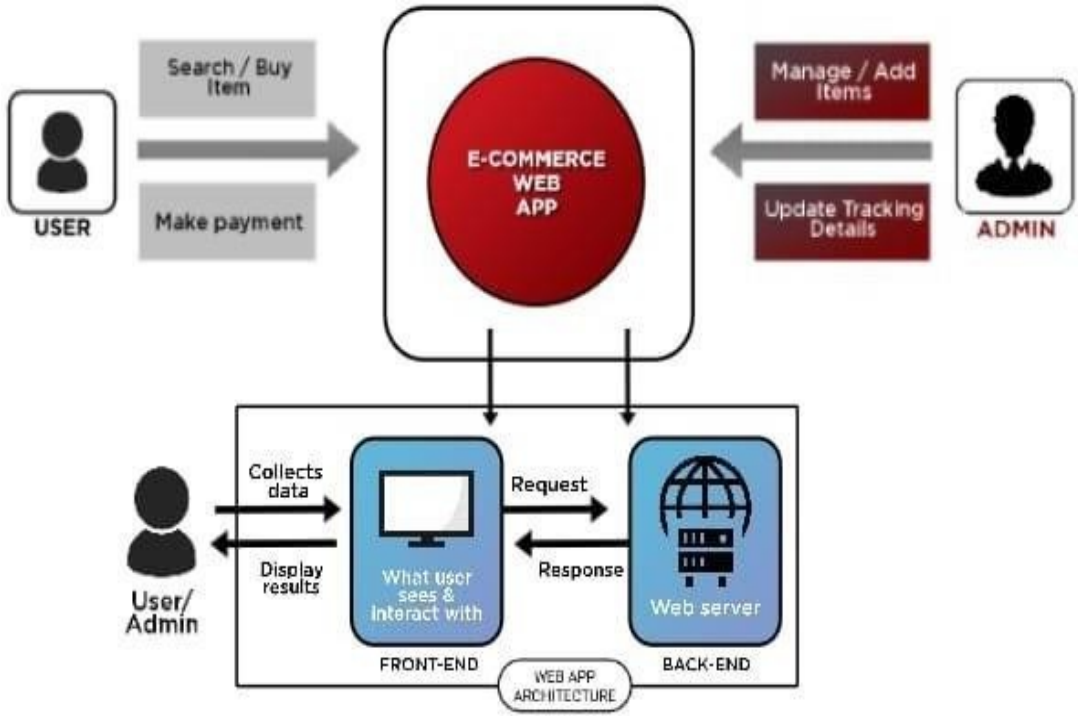

Fig4.3WebApplicationArchitecture

4.4 USECASEDIAGRAM:

4.4.1 CustomerSide:

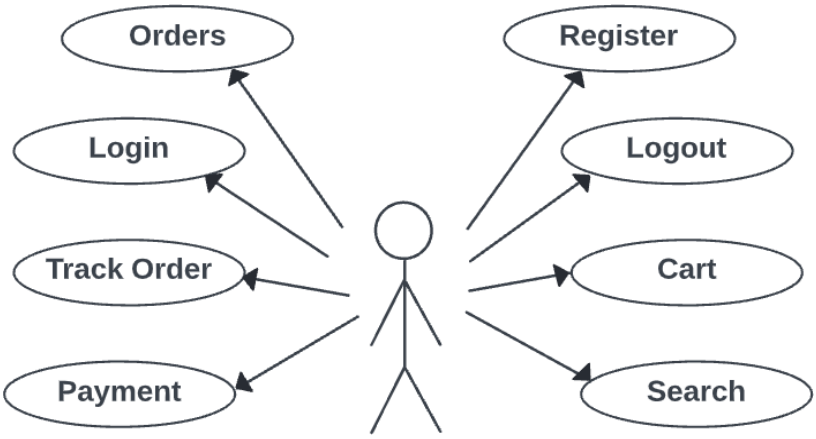

Fig4.4.1CustomerSide

4.4.2 AdminSide:

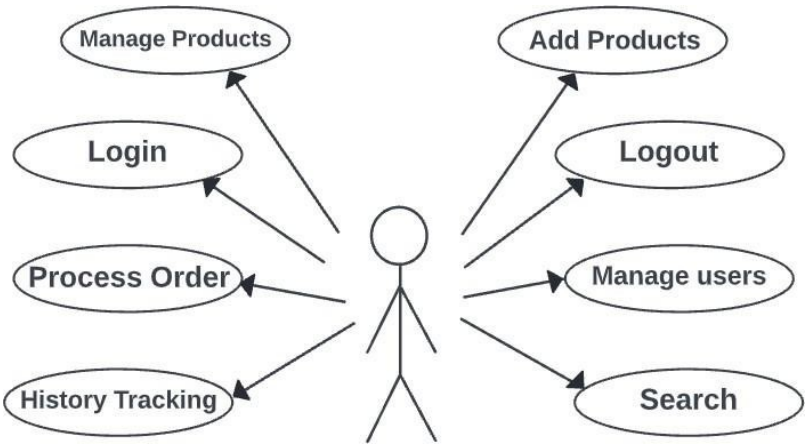

Fig4.4.2Admin Side

4.5 IMPLEMENTATION

4.5.1 CreatingtheVirtualEnvironment:

After python has been installed on the pc, a virtual environment module needs to be installed on the pc through python's package manager. The virtual environment makes it easy to run different versions of Django in isolation without interruptingtheprocessofoneanother.TheDjangomoduleisinstalledinthevirtual environment along with some other modules that are needed to develop the application specifying their versions.

Thisprocessisimplementedasfollows:

- Create Normal Project: Open the IDE and create a normal project by selecting File ->> New Project.
- InstallDjango: Next, we willinstall the Django module from the terminal. We will use PyCharm integrated terminal to do this task. One can also use cmd on windows to install the module by running python -m pip install django command
- Check Installed Django version: To check the installed Django version, you can run the python -m django -version command as shown below.
- Create Django Project: When we execute django-admin startproject command,thenitwillcreateaDjangoprojectinsidethenormalprojectwhich we already have created here. django-admin startproject OnlineShopping.
- Run Default Django web server: Django internally provides a default webserver where we can launch our applications. python manage.py runserver command in terminal. By default, the server runs on port 8000. Access the webserver at the highlighted URL.

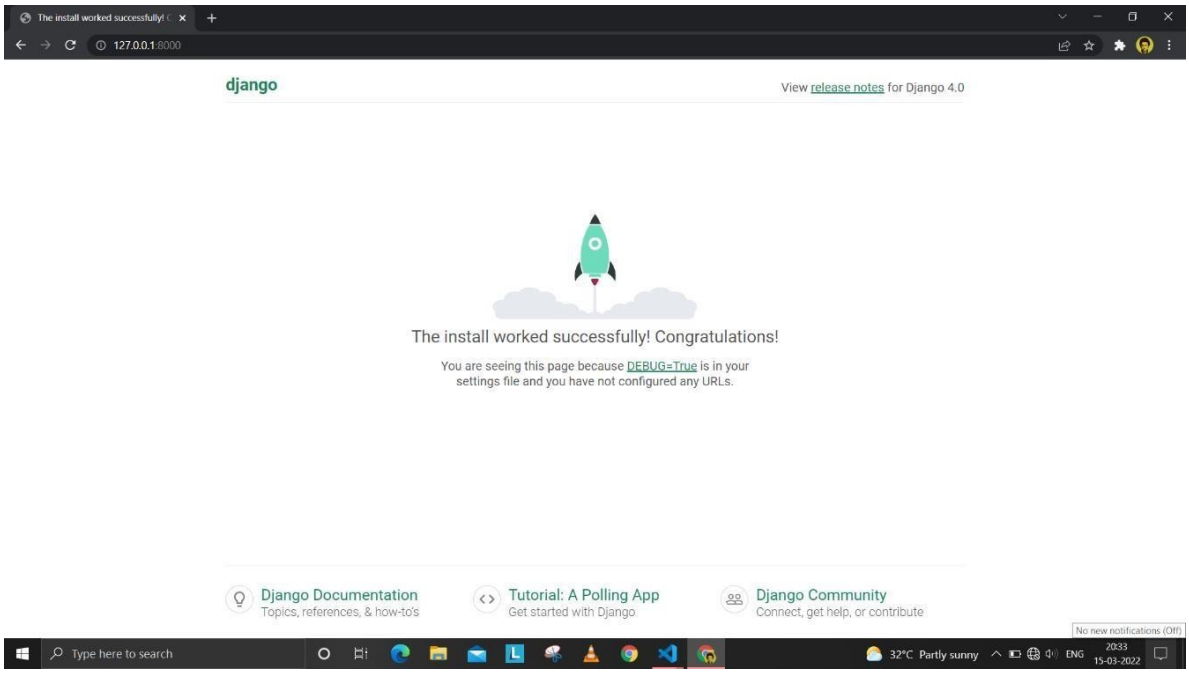

Fig4.5.1DjangoWebserver

4.5.2 CreatingvariousApps:

When the platform is ready for development, various applications can be created in the project which are relevant to the main idea of the website. For this research,theappscreated areasfollows:shop,cart,orders,andpayment.Eachof these apps comes with some important files when generated by the same Django- admin command in the command line of the operating system which allow programmers build the website easily and some are created by the programmer because it is not created by default by the command.

4.5.3 Creatingmodels:

A model is the single, definitive source of information about your data. It containstheessentialfieldsandbehaviorsofthedatayou’restoring.Eachattribute of the model represents a database field.

So,there areatotalof7modelscreatedforthisproject.

- Customermodel:Itsavesthebasic dataofthecustomerswhenthey register themselves.

- Productmodel:Itsavesthedataoftheproduct.The admincanaddanew product very easily using this model.
- Feature model: The admin can select the product and write any features aboutit.Andallthefeaturesofthatproductwillbevisibletotheuserswhen they view a specific product.
- Reviewmodel:Allthecustomerscanwriteareviewaboutaproductwhich the customers can read before buying it.
- Ordermodel:Itstoresthe orderdetailsaboutthecustomer,mainlythe order id.
- OrderItemsmodel:Itstorestheorderidofthecustomerfromtheorder model and the products with their quantity.
- CheckoutDetailsmodel:Itstoresmainlytheexactaddresswheretheorder is to be delivered.

4.5.4 CreatingaSuperuser(admin):

TheSuperuserissimplytheadminofthesite.Theadminaccountneedsto be created from the command line through the Django-admin command. The adminaccountmustbecreatedinordertomanagethesitewitha higherprivilege than the users of the site. The admin has the privilege to create, retrieve, update, anddeletedatacontentandusersfromthesitethroughtheadminsite.Allmodels present in the models.py file must be registered in admin.py file in order to allow the models to be visible for the admin.

After creating the models, we need to go to the admin panel to access the createdmodels.Hence,weneedasuperuserwhocanaccessthemodelsfromthe admin panel. The superuser can make any changes inside the models.

Forcreatingthesuperuserusethe**pythonmanage.pycreatesuperuser** command.

4.5.5 Creatingviewsforthemodels:

When the models have been successfully registered in the admin site, creating views for users is another task needed to be accomplished. This refers to thelogicalfunctionalitybetweentherequestandresponseoftheclientsandservers. There are two types of views:

- Function-based views: A view function, or view for short, is simply a Python function that takes a Web request and returns a Web response. This response can be the HTML contents of a Web page, or a redirect, or a 404 error,oranXMLdocument,oranimageoranything.Theviewitselfcontains whatever arbitrary logic is necessary to return that response. This code can live anywhere you want, as long as it's on your Python path. For the sake of putting the code somewhere, the convention is to put views in a file called views.py, placed in your project or application directory. The view function returns an HTML page that includes the request of the user.
- Class-Based views: Class-based views provide an alternative way to implement views as Python objects instead of functions. These allow you to structure your views and reuse code by harnessing inheritance and mixins. They do not replace function-based views, but have certain differences and advantages when compared to function-based views: Organization of code related to specific HTTP methods (GET, POST, etc.) can be addressed by separate methods instead of conditional branching. Object oriented techniques such as mixins (multiple inheritance) can be used to factor code into reusable components.

4.5.6 HowdoesDjango work?

TotrulyappreciateDjango,youwillneedtopeekunderthehoodandseethe various moving parts inside. This can be both enlightening and overwhelming. The numbered paths are as follows:

- The browser sends the request (essentially, a string of bytes) to the web server.
- The web server hands over the request to a WSGI server (say, uWSGI) or directly serves a file (say, a CSS file) from the filesystem.
- Unlikeawebserver,WSGIserverscanrunPythonapplications.Therequest populatesaPythondictionarycalledenvironand, optionally,passesthrough several layers of middleware, ultimately reaching your Django application.

- URLconfcontainedintheurls.pyofyour application selectsa viewto handle the request based on the requested URL. The request has turned into HttpRequest (a Python object).
- Theselectedviewtypically doesoneormore ofthefollowingthings:ItTalks to a database via the models. It Renders HTML or any other formatted response using templates. It Returns a plain text response (not shown). It Raises an exception.
- TheHttpResponseobjectgetsrenderedintoastring,asitleavestheDjango application.
- Abeautifullyrenderedwebpageisseeninyouruser'sbrowser.

4.6 APPLICATIONFUNCTIONALITIES

4.6.1 UserCreation

Django comes with a pre-built register form called UserCreationForm that connectstothepre-builtmodelUser.However,theUserCreationFormonlyrequires ausernameandpassword(password1istheinitialpasswordandpassword2isthe passwordconfirmation).Tocustomizethepre-builtform,firstcreateanewfilecalled forms.py in the app directory. This new file is created in the same directory as models.py and views.py. Then call UserCreationForm within a new class called NewUserForm and add another field called email. Save the email to the user. Add more fields as needed to the UserCreationForm.

4.6.2 Cookies

Djangoprovidesasessionframeworkthatletsyoustoreandretrievedataon a per -site-visitor basis. Django abstracts the process of sending and receiving cookies,byplacingasessionIDcookieontheclientside,andstoringalltherelated data on the server side. So the data itself is not stored client side.

4.6.3 SearchFunctionality

A common task for web applications is to search some data in the database withuserinput.Inasimplecase,thiscouldbefilteringalistofobjectsbyacategory. A more complex use case might require searching with weighting, categorization,

highlighting, multiple languages, and so on. This document explains some of the possible use cases and the tools you can use.

4.6.4 CartFunctionality

In Django-SHOP the cart's content is always stored inside the database. In previousversionsofthesoftware,thecart'scontentwaskeptinside the session for anonymous users and stored in the database for logged in users. Now the cart is always stored in the database. This approach simplifies the code and saves some random-accessmemory,butaddsanotherminorproblem:Fromatechnicalpointof view, the checkout page is the same as the cart. They can both be on separate pages, or be merged on the same page. Since what we would normally name the "Checkout Page", is only a collection of Cascade Plugins, we won't go into further detail here.

4.6.5 OrderFunctionality

During checkout, at a certain point the customer has to click on a button named "Place Order". This operation performs quite a few tasks: One of them is to convertthecartwithits items intoanorder.Thefinaltask is toresetthecart, which means to remove its content. This operation is atomic and not reversible.

4.6.6 ChangePasswordFunctionality

Djangodoesnotstoreraw(cleartext)passwordsontheusermodel,butonlyahash (see documentation of how passwords are managed for full details). Because of this,donotattemptto manipulatethepasswordattributeoftheuserdirectly.Thisis why a helper function is used when creating a user. To change a user's password, youhaveseveraloptions:manage.pychangepassword*username*offersamethod of changing a user's password from the command line.

- Itpromptsyouto changethepasswordofagivenuser whichyoumust enter twice.
- Iftheybothmatch,thenewpasswordwill bechangedimmediately.
- Ifyoudonotsupplyauser,thecommandwillattempttochangethepassword whose username matches the current system user.

4.6.7 ContactUsFunctionality

A contact form is a common feature of many websites that provides a way for users to get in touch with the site's administrators without having to open their email or hop on the phone. In a Python Django application, a contact form can be used to store a user's contact information in the site's database.

4.7 TESTING

To affirm the E-commerce store assessments had been made at distinct stagesofthetasks.Wecheckedthereliabilityofallofthefunctions.Thetestisbuilt on the customer/user and admin side. The customer test proved that an account couldbecreated,logincanbeestablished,thecartcanbeloadedwithproductsand the customer can check out when done shopping. The administrator can login into the admin panel afterwards. The admin can then manage all the contents in the store.

The customer can only view the products, if he has logged in as a user. Without being a customer/user, he is unable to viewany products which are shown on the home page. Some quantity of products was added into the shopping cart, and then I proceeded to checkout. After successful checkout, the cart became empty.ThisindicatesthatthecartworksappropriatelyasitshouldandtheOrderID is shown in a pop-up. The customer can manipulate his cart, such as updating the cart or adding a product to cart. The search bar shows the results of the product search.Iftheuser didn'ttypeanything,amessagewill bedisplayedsayingthatthe user forgot to type.

CHAPTER–5

5. RESULTSANDPERFORMANCEANALYSIS

5.1 ADMINISTRATIONPANEL

Theadmincanaddtheproducttothewebsite,wherehewillgivetheproduct name, product price and image, then save it and that product displayed in the website. These product data are stored has tables in Django default database SQLite.

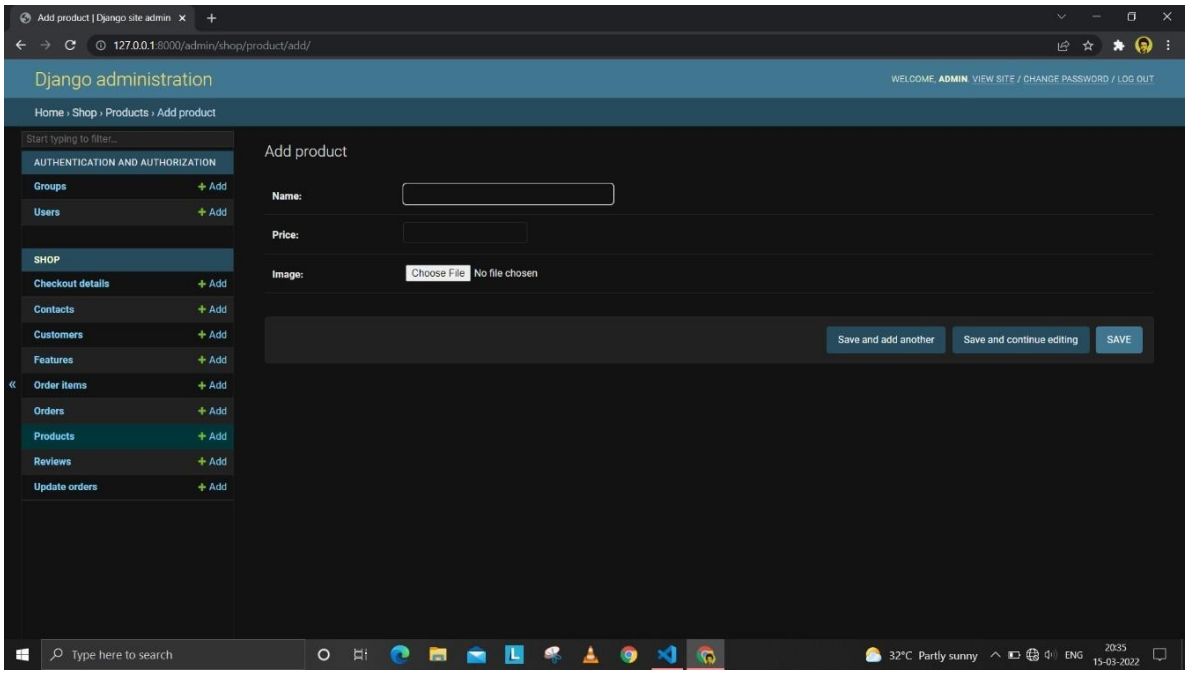

Fig5.1AddproductfeatureinDjangoAdministrationpanel

5.2 HOMEPAGEBEFORELOGIN

The home page is displayed successfully with all the products, navigation bar with search functionality and footer at the bottom. If you want to purchase you have to login as a customer. If you don't have an account then click register. It will redirect to register page is displayed successfully with user registration form fields and an URL which directs them to login page. The login page is displayed successfully in order to login users can type their username and password.

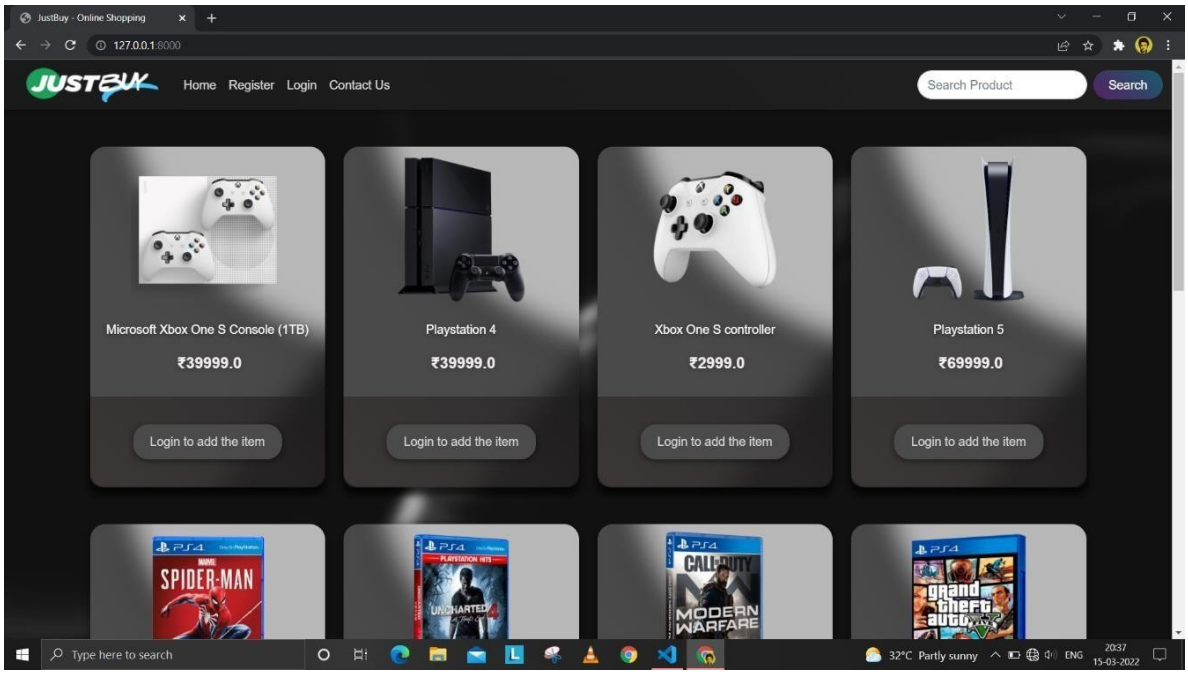

Fig5.2Homepagewithaddedproducts

5.3 HOMEPAGEAFTERLOGIN

After the user is logged in successfully, the cart will be displayed in the navigation bar which indicates that they can now view the products and add the products to the cart. The products selected by the user will be shown in the order summary with quantity increase and decrease functionality. For further process, checkout button is available. The checkout page is shown successfully with the productsummary whichwasfinalizedbythecustomer.Underthis,checkoutdetails will be displayed with form fields required to make shipment and selection of payment mode will be available. After the order is placed, a pop-up containing the Order ID is displayed and then the user will be redirected to home page where the cart becomes empty. In order to track the progress of the orders which was placed by the customer, you can check the, in the Track order page by entering a valid Order ID in the track order text field. If the user faces any problems or issues, they canreportthatissuesinthecontactuspage, whichis available inthe navbar.After enteringthemessageinthedescriptionformfield,apop-upisdisplayed

successfullywhichindicatesthatthemessagewassubmittedtotheadmin.To report a problem or to ask any queries, login is not mandatory.

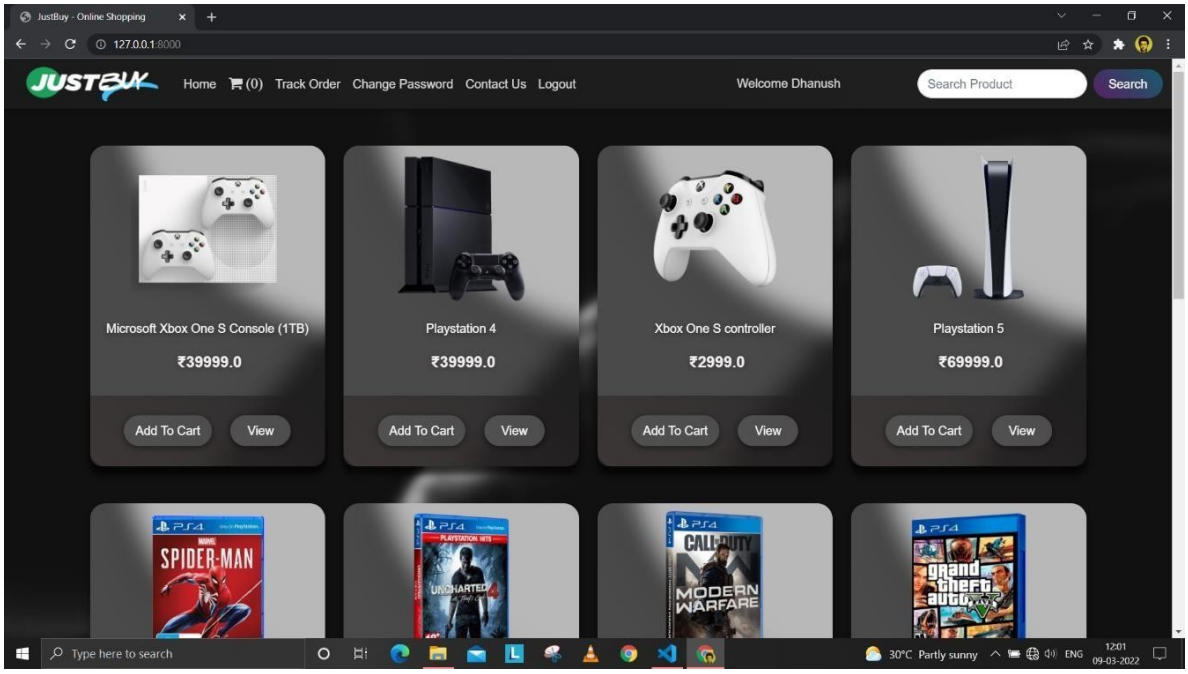

Fig5.3 Homepageafterlogin

CHAPTER–6

5. CONCLUSIONANDFUTUREWORKS

6.1 CONCLUSION

E-commerce is continuously progressing and is becoming more and more important to business as technology continuous to advance and is something that should be taken advantage of and implemented. This E-commerce platform is designed to provide a web-based application that would make searching, viewing and selection of a product is easier.

The user can search for a product interactively and the search enginerefine the products available based on the user's input. Then the user can view the full specificationsandselecttheproducts,theusercanseetheproductsinthecartand proceeds to checkout where they enter the address details and select the mode of payment. The administrator can verify the orders. However, the customer can still look at their status of the orders in the Track us page using the order ID.

With this platform, more opportunities will be created for profit and advancements for businesses, while creating more options for both the consumers and sellers.

6.2 FUTUREWORKS

- Separate invoices need other than order summary details need to be implemented.
- Userprofilecustomization needtobeadded.
- Emailsandnotificationsneedtobesenttocustomersfornewarrivalsor discounts.
- Categorizingofallproductsshouldbeimplemented.
- Therehavetobelanguagevarietiessothatnon-English usersandcustomers can

APPENDIX

A.SOURCECODE

MODELS.PY

```
fromdjango.dbimportmodels

fromdjango.contrib.auth.modelsimportUser

from django.utils.timezone import now


classCustomer(models.Model):

    user=models.OneToOneField(User,on_delete=models.CASCADE)

    name = models.CharField(max_length=100)

    email=models.CharField(max_length=100)

    phone_number=models.CharField(max_length=10,null=True,blank=True)


    def___str(self):

        returnstr(self.user)


classProduct(models.Model):

    name=models.CharField(max_length=100)

    price = models.FloatField()

    image=models.ImageField(upload_to="",default="")


    def___str(self):

        returnself.name
```

```
classFeature(models.Model):
    product=models.ForeignKey(Product,on_delete=models.CASCADE)
    feature=models.CharField(max_length=1000,null=True,blank=True)

    def___str(self):
        returnstr(self.product)+"Feature:"+self.feature

classReview(models.Model):
    customer=models.ForeignKey(Customer,on_delete=models.CASCADE)
    product = models.ForeignKey(Product, on_delete=models.CASCADE)
    content = models.TextField() datetime=models.DateTimeField(default=now)

    def___str(self):
        returnstr(self.customer)+"Review:"+self.content

classOrder(models.Model):
    customer=models.ForeignKey(Customer,on_delete=models.SET_NULL,
null=True)
    date_ordered = models.DateTimeField(default=now)
    complete = models.BooleanField(default=False)

    def___str(self):
        returnstr(self.id)
```

@property

```
    defget_cart_total(self): orderitems=self.orderitem_set.all()
        total=sum([item.get_totalforiteminorderitems]) return
        total

    @property
    defget_cart_items(self):
        orderitems=self.orderitem_set.all()
        total=sum([item.quantityforiteminorderitems]) return
        total

classOrderItem(models.Model):
    product=models.ForeignKey(Product,on_delete=models.SET_NULL,
null=True)
    order=models.ForeignKey(Order,on_delete=models.SET_NULL,null=True) quantity =
    models.IntegerField(default=0)
    date_added=models.DateTimeField(default=now)

    def___str(self):
        returnstr(self.order)

    @property
    defget_total(self):
        total=self.product.price*self.quantity return
        total
```

```
classUpdateOrder(models.Model):
    order_id=models.ForeignKey(Order,on_delete=models.CASCADE)
    desc = models.CharField(max_length=500)
    date=models.DateField(default=now)

    def___str(self):
        returnstr(self.order_id)

classCheckoutDetail(models.Model):
    customer=models.ForeignKey(Customer,on_delete=models.SET_NULL,
null=True)
    order=models.ForeignKey(Order,on_delete=models.SET_NULL,null=True)
    phone_number = models.CharField(max_length=10, blank=True, null=True)
    total_amount = models.CharField(max_length=10, blank=True,null=True)
    address = models.CharField(max_length=300)
    city = models.CharField(max_length=100)
    state = models.CharField(max_length=100)
    zipcode=models.CharField(max_length=100)
    payment=models.CharField(max_length=100,blank=True)
    date_added = models.DateTimeField(default=now)

    def___str(self):
        returnself.address

classContact(models.Model):
```

```
name=models.CharField(max_length=100)
```

```
    email = models.CharField(max_length=50)
    phone = models.CharField(max_length=10)
    desc=models.CharField(max_length=1000)

    def___str(self):
        returnself.name
```

VIEWS.PY

```
fromdjango.http.responseimportHttpResponse
from django.shortcuts import render, redirect
from .models import *
fromdjango.httpimportJsonResponse
import json
fromdjango.contrib.auth.modelsimportUser
fromdjango.contrib.authimportauthenticate,login,logout from .
inherit import cartData

defindex(request):
    data=cartData(request)
    items = data['items']
    order = data['order']
    cartItems=data['cartItems']

    products=Product.objects.all()
    returnrender(request,"index.html",{'products':products,'cartItems':cartItems})
```

```
defcart(request):
    data=cartData(request)
    items = data['items']
    order = data['order']
    cartItems=data['cartItems']
    try:
        cart=json.loads(request.COOKIES['cart'])
    except:
        cart = {}
    print('Cart:',cart)

    foriincart: try:
            cartItems+=cart[i]["quantity"]

            product=Product.objects.get(id=i)
            total=(product.price*cart[i]["quantity"])

            order["get_cart_total"] += total
            order["get_cart_items"]+=cart[i]["quantity"]

            item={
                'product':{
                    'id':product.id,
```

```
                'name':product.name,
                'price':product.price,
                'image':product.image,
            },
            'quantity':cart[i]["quantity"],
            'get_total':total
        }
        items.append(item)
    except:
        pass
returnrender(request,"cart.html",{'items':items,'order':order,
'cartItems':cartItems})


defcheckout(request):
    data=cartData(request)
    items = data['items']
    order = data['order']
    cartItems=data['cartItems']
    total = order.get_cart_total
    if request.method == "POST":
        address=request.POST['address']
        city = request.POST['city']
        state = request.POST['state']
        zipcode=request.POST['zipcode']
        phone_number=request.POST['phone_number']
```

```
payment = request.POST['payment']
```

```
        shipping_adress=CheckoutDetail.objects.create(address=address,city=city,
phone_number=phone_number, state=state, zipcode=zipcode,
customer=request.user.customer, total_amount=total, order=order, payment=payment)

        shipping_adress.save()

        iftotal==order.get_cart_total:

            order.complete = True

        order.save()

        id  =order.id

        alert = True

        returnrender(request,"checkout.html",{'alert':alert,'id':id})

    returnrender(request,"checkout.html",{'items':items,'order':order, 'cartItems':cartItems})


defupdateItem(request):

    data=json.loads(request.body)

    productID = data['productID']

    action = data['action']


    print('Action:', action)

    print('productID:',productID)


    customer=request.user.customer

    product=Product.objects.get(id=productID)

    order,created=Order.objects.get_or_create(customer=customer,
complete=False)

    orderItem,created=OrderItem.objects.get_or_create(order=order,
```

product=product)

```
    update_order,created=UpdateOrder.objects.get_or_create(order_id=order, desc="Your
Order is Successfully Placed.")


    ifaction =='add':

        orderItem.quantity=(orderItem.quantity+1)

    elif action == 'remove':

        orderItem.quantity=(orderItem.quantity-1)


    orderItem.save()

    update_order.save()


    iforderItem.quantity<=0:

        orderItem.delete()

    returnJsonResponse('Itemwasadded',safe=False)


def    product_view(request,    myid):

    customer=request.user.customer

    product   =   Product.objects.filter(id=myid).first()

    feature=Feature.objects.filter(product=product)

    reviews=Review.objects.filter(product=product)

    data = cartData(request)

    items=data['items']

    order=data['order']

    cartItems=data['cartItems']
```

```
ifrequest.method=="POST":
```

```
        content=request.POST['content']

        review=Review(customer=customer,content=content,product=product)

        review.save()

        returnredirect(f"/product_view/{product.id}")

    returnrender(request,"product_view.html",{'product':product,
'cartItems':cartItems, 'feature':feature, 'reviews':reviews})


defsearch(request):

    data=cartData(request)

    items = data['items']

    order = data['order']

    cartItems=data['cartItems']

    if request.method == "POST":  search=request.POST['search']

        products=Product.objects.filter(name___contains=search)

        returnrender(request,"search.html",{'search':search,'products':products,
'cartItems':cartItems})

    else:

        returnrender(request,"search.html")



defchange_password(request):

    ifnotrequest.user.is_authenticated:

        return redirect('/login')

    data=cartData(request)

    items = data['items']
```

```
    order = data['order']
    cartItems=data['cartItems']
      ifrequest.method =="POST":
        current_password=request.POST['current_password']
        new_password = request.POST['new_password']
        try:
            u = User.objects.get(id=request.user.id)
            ifu.check_password(current_password):
                u.set_password(new_password)
                u.save()
                alert=True returnrender(request,"change_password.html",{'alert':alert})
                else:
                  currpasswrong=True
                returnrender(request,"change_password.html",
{'currpasswrong':currpasswrong})
        except:
            pass returnrender(request,"change_password.html",{'cartItems':cartItems})


defcontact(request):
    if request.method=="POST":
        name = request.POST['name']
        email = request.POST['email']
        phone=request.POST['phone']
        desc = request.POST['desc']
```

```
        contact=Contact(name=name,email=email,phone=phone,desc=desc)
        contact.save()
        alert=True returnrender(request,'contact.html',{'alert':alert})
    return  render(request,  "contact.html")

defloggedin_contact(request):
    data = cartData(request)
    items = data['items']
    order = data['order']
    cartItems = data['cartItems']
    ifrequest.method=="POST":
        name =request.user
        email=request.user.email
        phone = request.user.customer.phone_number
        desc = request.POST['desc']
        contact=Contact(name=name,email=email,phone=phone,desc=desc)
        contact.save()
        alert=True returnrender(request,'loggedin_contact.html',{'alert':alert})
    returnrender(request,"loggedin_contact.html",{'cartItems':cartItems})

deftracker(request):
    ifnotrequest.user.is_authenticated:
        return redirect('/login')
```

```
    data=cartData(request)

    items = data['items']

    order = data['order']

    cartItems=data['cartItems']

    if request.method == "POST":

        order_id=request.POST['order_id']

        order = Order.objects.filter(id=order_id).first()

        order_items=OrderItem.objects.filter(order=order) update_order =

        UpdateOrder.objects.filter(order_id=order_id) print(update_order)

        returnrender(request,"tracker.html",{'order_items':order_items,
'update_order':update_order})

    returnrender(request,"tracker.html",{'cartItems':cartItems})



defregister(request):

    ifrequest.user.is_authenticated:

        return redirect("/") else:

        ifrequest.method=="POST":

            username = request.POST['username']

            full_name=request.POST['full_name']

            password1=request.POST['password1']

            password2=request.POST['password2']

            phone_number=request.POST['phone_number']
```

```
email = request.POST['email']
```

```
        ifpassword1!=password2:
            alert = True
            returnrender(request,"register.html",{'alert':alert})
        user=User.objects.create_user(username=username,
password=password1, email=email)
        customers=Customer.objects.create(user=user,name=full_name,
phone_number=phone_number, email=email)
        user.save()
        customers.save()
        alert = True
        returnrender(request,"success.html",{'alert':alert})
    return render(request, "register.html")


defLogin(request):
    ifrequest.user.is_authenticated:
        return redirect("/") else:
        ifrequest.method=="POST":
            username=request.POST['username']
            password = request.POST['password']
            user=authenticate(username=username,password=password)


            if user is not None:
                login(request,user)
                return redirect("/")
```

```
        else:
            alert=True
            returnrender(request,"login.html",{"alert":alert})
    return render(request, "login.html")


defLogout(request):
    logout(request)
    alert = True
    returnrender(request,"index.html",{'alert':alert})
```

BASE.HTML:

```
<!doctypehtml>
<htmllang="en">


<head>
    <metacharset="utf-8">
    <metaname="viewport"content="width=device-width,initial-scale=1">


    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/css/bootstrap.min.css
"rel="stylesheet"
        integrity="sha384-
EVSTQN3/azprG1Anm3QDgpJLIm9Nao0Yz1ztcQTwFspd3yD65VohhpuuCO
mLA   SjC" crossorigin="anonymous">
        <script src="https://kit.fontawesome.com/90ccb65d9b.js"
crossorigin="anonymous"></script>
        <linkrel="preconnect"href="https://fonts.googleapis.com">
```

```
        <linkrel="preconnect"href="https://fonts.gstatic.com"crossorigin>

        <link
href="https://fonts.googleapis.com/css2?family=Red+Hat+Display:wght@500&disp
lay=swap" rel="stylesheet">



    <title>{%blocktitle%}{%endblock%}</title>

</head>



<body>

    <navclass="navbarnavbar-expand-lgnavbar-darksticky-top">

        <divclass="container-fluid">

            <aclass="navbar-brand"href="/"><imgid="logonav"
src="/static/logo2.png"></a>

            <buttonclass="navbar-toggler"type="button"data-bs-toggle="collapse"

                data-bs-target="#navbarSupportedContent" aria-
controls="navbarSupportedContent"aria-expanded="false"

                aria-label="Toggle navigation">

                <spanclass="navbar-toggler-icon"></span>

            </button>

            <divclass="collapsenavbar-collapse"id="navbarSupportedContent">

                <divclass="containerw-75">

                <ulclass="navbar-navme-automb-2mb-lg-0">

                    <li>

                        <aclass="nav_links"href="/">Home</a>

                    </li>

                    {%ifrequest.user.is_authenticated %}
```

<li>

```
                <aclass="nav_links"href="/cart/"><spanclass="fafa-shopping-
cart"></span>({{cartItems}})</a>
            </li>
            <li>
                <aclass="nav_links"href="/tracker/">TrackOrder</a>
            </li>
            <li>
                <aclass="nav_links"href="/change_password/">Change
Password</a>
            </li>
            <li">
                <aclass="nav_links"href="/loggedin_contact/">ContactUs</a>
            </li>
            {%endif%}
            {%ifnotrequest.user.is_authenticated%}
            <li>
                <aclass="nav_links"href="/register/">Register</a>
            </li>
            <li>
                <aclass="nav_links"href="/login/">Login</a>
            </li>
            <li>
                <aclass="nav_links"href="/contact/">ContactUs</a>
            </li>
            {%else%}
```

<aclass="nav_links3"href="/logout/">Logout</a>

```
                <aclass="nav_links2"href="#">Welcome{{request.user}}</a>


            {%endif%}

        </ul>

        </div>

        <formclass="d-flex" method="POST"action="/search/">{%csrf_token
%}

            <input class="form-control me-2" type="search" name="search"
placeholder="SearchProduct"aria-label="Search"style="border-radius:40px;">

            <buttonhref="/search/"class="btn-btn-secondary"
type="submit">Search</button>

        </form>

      </div>

    </div>

  </nav>



  {%blockbody%}

  {%endblock %}

  <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/js/bootstrap.bundle.min.js"

    integrity="sha384-
MrcW6ZMFYlzcLA8Nl+NtUVF0sA7MsXsP1UyJoMp4YLEuNSfAP+JcXn/tWtIa
xVX M"

    crossorigin="anonymous"></script>

  <script src="https://code.jquery.com/jquery-

    3.4.1.slim.min.js" integrity="sha384-
J6qa4849blE2+poT4WnyKhv5vZF5SrPo0iEjwBvKU7imGFAV0wwj1yYfoRSJoZ+n "

    crossorigin="anonymous"></script>
```

```
<script>
    varuser ='{{request.user}}'


    functiongetToken(name){ va
        r cookieValue = null;
        if(document.cookie&&document.cookie!==''){ var
            cookies = document.cookie.split(';'); for(vari=0;i
            <cookies.length;i++){ var
                cookie = cookies[i].trim();
                //Doesthiscookiestringbeginwiththenamewewant?
                if(cookie.substring(0,name.length+1)===(name+'=')){ cookieValue
                    =
decodeURIComponent(cookie.substring(name.length+1));
                    break;
                }
            }
        }
        returncookieValue;
    }
    varcsrftoken =getToken('csrftoken');


    functiongetCookie(name){ varcookieArr=d
        ocument.cookie.split(";");
```

```
for(var i =0;i<cookieArr.length;i++){
```

```
        varcookiePair=cookieArr[i].split("=");


        if(name==cookiePair[0].trim()){
            //Decodethecookievalueandreturn
            returndecodeURIComponent(cookiePair[1]);
        }
    }


    //Returnnullifnotfound return
    null;
}
varcart =JSON.parse(getCookie('cart'))


if(cart==undefined){ c art
    = {}
    console.log('CartCreated!',cart)
    document.cookie='cart='+JSON.stringify(cart)+";domain=;path=/"
}
console.log('Cart:',cart)


varupdateBtns=document.getElementsByClassName('update-cart')


for (var i = 0; i < updateBtns.length; i++)
    { updateBtns[i].addEventListener('click',function(){ varproductI
        D =this.dataset.product
```

```
        var action = this.dataset.action
        console.log('productId:',productID,'action:',action)
        console.log('USER:', user)
        if (user == 'AnonymousUser')
            { addCookieItem(productID, action)
        } else{
            updateUserOrder(productID,action)
        }
    })
}

functionaddCookieItem(productID,action){ console.lo
    g('Not logged in')
    if(action =='add'){
          if (cart[productID] == undefined)
            { cart[productID]={'quantity':1}

        } else{
            cart[productID]['quantity']+=1
        }
    }

    if (action == 'remove')
        {  cart[productID]['quantity']-=1
```

```
        if (cart[productID]['quantity'] <= 0)
            { console.log('Itemshouldbedeleted')
            delete cart[productID];
        }
    }
    console.log('Cart:',cart)
    document.cookie='cart='+JSON.stringify(cart)+";domain=;path=/"
    location.reload()
}

function updateUserOrder(productID, action)
    { console.log('Userisloggedin,sendingdata...')
    var url = '/update_item/'
    fetch(url,
        {method:'POST',
        headers: {
            'Content-Type':'application/json',
            'X-CSRFToken': csrftoken,
        },
        body:JSON.stringify({'productID':productID,'action':action})
    })
        .then((response) =>
            { returnresponse.json ()
        })
```

```
            .then((data) =>
                { console.log('data:',data)
                location.reload()
            })
        }

    </script>
    {%blockjs%}
    {%endblock %}

</body>

</html>
```

INDEX.HTML:

```
{%extends'base.html'%}
{%loadstatic%}
{%blocktitle%}JustBuy-OnlineShopping {%endblock%}
{%blockcss%}
{%endblock %}
{%blockbody%}


<divclass="containermb-5mt-3">
```

<divclass="row">

```
    {%forproductinproducts %}
    <divclass="col-md-3my-3">
      <divclass="cardmt-3">
        <divclass="product-1align-items-centerp-2text-center">
          <imgsrc="/media/{{product.image}}"alt=""class="rounded"            width="200"
height="200">
          <h5class="name-h5">{{product.name}}</h5>


          <divclass="costmt-3text-dark">
            <h4class="name-h4"> ₹{{product.price}}</h4>
          </div>


        </div>


            <divclass="p-3squaretext-centertext-whitemt-3cursor">
          {%ifrequest.user.is_authenticated%}
          <buttondata-product="{{product.id}}"data-action="add"class="btn-add add- btn
update-cart">Add To Cart</button>
          <ahref="/product_view/{{product.id}}/"class="btn-view">View</a>
          {%else%}
           <ahref="login/"class="btn-login">Logintoaddtheitem</a>
          {%endif%}
        </div>
      </div>
    </div>
    {%endfor%}
  </div>
</div>
```

```
<footerclass="footer-distributed">
  <spanstyle="font-family:'RedHatDisplay',sans-serif">

    <divclass="footer-left">
  <imgid="logonav2"src="/static/logo2.png">

        <pid="footercompanyname"><br>©2022E-CommercePvt.Ltd.</p>
    </div>

    <divclass="footer-center">
        <div>
            <iclass="fafa-map-marker"></i>
              <p><span>785 - Rupa Square,
                ShopNo:X- 96,Sector -1</span> T-
                Nagar, Chennai - 632145</p>
        </div>

        <div>
            <iclass="fafa-phone"></i>
            <p>+9145-86321459</p>
        </div>
        <div>
            <iclass="fafa-envelope"></i>
            <p><a                                    href="mailto:support@justbuy.com"
style="color:#e668ff;">support@justbuy.com</a></p>
        </div>
```

```
</div>
<divclass="footer-right">
    <pclass="footer-company-about">
        <span>About the company</span>
        JustBuyisanE-commerceplatform.</p>
        <ulclass="icon-container">
          <liclass="icon-list">
            <aclass="icon-link"href="#">
              <iclass="fabfa-facebook"></i>
            </a>
          </li>
          <liclass="icon-list">
            <aclass="icon-link"href="#">
              <iclass="fabfa-twitter"></i>
            </a>
            </li>
          <liclass="icon-list">
            <aclass="icon-link"href="#">
              <iclass="fabfa-instagram"></i>
            </a>

          </li>
          <liclass="icon-list">
            <aclass="icon-link"href="#">
              <iclass="fabfa-youtube"></i>
            </a>
          </li>
```

```
            </ul>
          </span>
    </div>

</footer>

{%endblock %}
{%blockjs%}
<script>

{%ifalert%}
  alert("LogoutSuccessful.")
  window.location.href = '/'
{%endif%}
</script>
{%endblock %}
```